

进程通信, 理论课作业 参考答案 (部分)

同济大学计算机系操作系统作业 2026 拔尖班、国豪班

2026/4/23

姓名

学号

一、选择、填空

(一) 临界区、临界资源

1. 临界区是指_____。
A. 并发进程中用于实现进程互斥的程序段
B. 并发进程中用于实现进程同步的程序段
C. 并发进程中用户实现进程通信的程序段
D. 并发进程中访问共享变量的程序段
2. 相关临界区是指_____。
A. 一个独占资源
B. 并发进程中与共享变量有关的程序段
C. 一个共享资源
D. 并发进程中涉及相同变量的那些程序段
3. 管理若干进程共享某一资源的相关临界区应满足三个要求, 其中不考虑_____。
A. 一个进程可以抢占已分配给另一进程的资源
B. 任何进程不应该无限地逗留在它的临界区中
C. 一次最多让一个进程在临界区执行
D. 不能强迫一个进程无限地等待进入它的临界区
注释: 参考互斥机制应该遵循的原则。Part5 进程通信 1, 第 18 页。
4. 若一个进程已进入临界区, 其它欲进入相关临界区的进程必须___等待___。

(二) 信号量 和 P、V 操作

1. _____是一种只能进行 P 操作和 V 操作的特殊变量。
A. 调度
B. 进程
C. 同步
D. 信号量
2. P、V 操作是_____。
A. 两条低级进程通信原语
B. 两组不同的机器指令
C. 两条系统调用命令
D. 两条高级进程通信原语
3. 若 P、V 操作的信号量 S 初值为 2, 当前值为 -1, 则表示有_____等待进程。
A. 0 个
B. 1 个
C. 2 个
D. 3 个
4. 用 P、V 操作管理互斥资源时, 信号量的初值应定义为_____。
A. -1
B. 0
C. 1
D. 任意值

- 5、用 V 操作唤醒一个等待进程时，被唤醒进程的状态变为_____。
A.等待 B. 就绪 C. 运行 D. 完成
- 6、对于整型信号量，在执行一次 P 操作时，信号量的值应_____。
A. 不变 B. 加 1 C. 减 1 D. 减指定数值
- 7、在执行 V 操作时，当信号量的值_____时，应释放一个等待该信号量的进程。
A. >0 B. <0 C. >=0 D. <=0
- 8、P、V 操作必须在屏蔽中断下执行，这种不可被中断的过程称为_____。
A. 初始化程序 B. 原语（原子操作） C. 子程序 D. 控制模块
- 9、信号量的物理意义是当信号量值大于零时表示 可用资源的数目；当信号量值小于零时，其绝对值为 等待使用资源的进程数目。
- 10、用 P、V 操作管理相关临界区时，必须成对出现，任何一个进程在进入临界区之前应调用 P 操作，退出临界区时应调用 V 操作。
- 11、有 m 个进程互斥共享同一临界资源，若使用信号量机制实现对临界资源的互斥访问，则信号量值的变化范围是 [-(m-1), 1]。
- 12、操作系统中，对信号量 S 的 P 原语操作定义中，使进程进入相应等待队列等待的条件是 S.value <= 0。（-1 之前）
- 13、若信号量的初值为 n，用 PV 操作能限制一次_____进程进入临界区操作。语义不清，不做
- 14、用 PV 操作实现进程同步时，调用 P 操作 测试消息是否到达，调用 V 操作 发送消息。
- 15、信号量 S 保护共享资源 R。S 的初值是 2，当前值为-1。请问资源 R 有 2 个？当前， 1 个进程正在等待使用 R 资源。

（三）进程间的同步和互斥

- 1、进程间的同步是指进程间在逻辑上的相互_____关系。
A.联接 B. 制约 C. 继续 D. 调用
- 2、进程间的互斥与同步分别表示了各进程间的_____。
A. 竞争与协作 B. 相互独立与相互制约
C. 不同状态 D. 动态性与并发性
- 3、实现进程互斥时，用_____对应，对同一个信号量调用 P、V 操作实现互斥。
A. 一个信号量与一个临界区 B. 一个信号量与一个相关临界区
C. 一个信号量与一组相关临界区 D. 一个信号量与一个消息
- 4、实现进程同步时，每一个消息与一个信号量对应，进程_____可把不同的消息发送出去。

- A. 在同一信号量上调用 P 操作 B. 在不同信号量上调用 P 操作
C. 在同一信号量上调用 V 操作 D. 在不同信号量上调用 V 操作

5、进程的互斥是指当有若干进程都要使用某一共享资源时，任何时刻最多只允许一个进程去使用。

6、进程的同步是指并发进程之间存在一种制约关系，一个进程的执行依赖另一个进程的消息（或，一个任务执行完毕，另一个任务才能执行）。

7、进程的互斥实质上是进程同步的一种特殊情况。

8、两个进程访问同一个共享变量，怎样实现把共享变量放在文件或共享内存里
如果未加控制，会出现什么情况？值会出错
保证变量的值不出错，应该怎样做互斥（上锁）
为此，计算机系统为应用系统提供了一些工具。最常用的是信号量，自旋锁
在编写应用系统的时候，如果没做好互斥，会出现什么情况共享数据结构值会出错
同步没做好呢？系统时序不对，无法维护前驱、后继关系。可能会发生死锁或进程饿死现象。

（四）与时间有关的错误（或称为竞争条件 race condition）

1. 只要涉及相同变量的若干进程的相关临界区执行时段没有交集，就不会造成与时间有关的错误。

[注释] 时间有关的错误是指多个进程同时对同一个共享变量进行访问时，变量最终的值取决于进程执行的先后次序，有时正确、有时错误。参见下面的例子：

例：假设某游乐场有一个入口和一个出口，管理人员需要随时了解当前游乐场中有多少游客，进程IN和进程OUT分别承担在入口处和出口处统计游客数的任务。当前游客数存放在共享计数器变量count中。

进程IN: count++; // 汇编之后的指令: 1: mov eax, count; 2: inc eax; 3: mov count, eax
进程OUT: count--; // 汇编之后的指令: 4: mov eax, count; 5: dec eax; 6: mov count, eax

T时刻，count的值是1000。有一个游客进入游乐场，同时，一名游客离开。
正确的系统状态应该是这6条指令执行完毕之后，count == 1000。

推进顺序1: 1 4 5 6 2 3。count的值，最终会是1001，减法操作的结果丢了。
推进顺序2: 4 5 1 2 3 6。count的值，最终会是999，加法操作的结果丢了。
推进顺序3: count++完整完成，之后再执行count--操作。或者，count--操作先完成，再执行count++操作。

这种变量最终的值和进程推进速度有关的错误叫做竞争条件(race condition)，又叫与时间有关的错误

二、简答题

- 1、简述临界区和相关临界区的概念。管理相关临界区有什么要求？

答：临界区是指并发进程中与共享变量有关的程序段。相关临界区是指并发进程中涉及到相同变量的那些程序段。管理相关临界区有三点要求：(1) 一次最多让一个进程在临界区执行；
(2) 任何一个进入临界区执行的进程必须在有限的时间内退出临界区；(3) 不能让进程无限等待进入临界区。

2、为什么并发进程执行时可能会产生与时间有关的错误？如何避免？

答：进程在共享数据结构使用期间响应中断放弃 CPU。

如何避免？序列化共享数据结构的访问操作（就是一个进程访问完毕，另一个进程再访问）

3、请给出 PV 操作的定义。假设 PV 操作用信号量 s 管理某个共享资源，请问当 $s.value > 0$ ， $s.value = 0$ 和 $s.value < 0$ 时，它们的物理意义是什么？

答：P 操作是同步获得共享资源的访问权限。

V 操作是共享资源使用完毕，释放访问权限。

记信号量为 s ， $P(s)$ 和 $V(s)$ 定义如下：

```
procedure P(s)
    s.value:=s.value-1;
    if s.value<0
        sleep(s);
```

```
procedure V(s)
    s.value:=s.value+1;
    if s.value<=0
        wakeup (s);
```

> 0 ，资源还有。 $s.value$ 是空闲资源数量。

$= 0$ ，资源用完了，但没有阻塞进程。

< 0 ，资源用完了， $|s.value|$ 是等待使用资源的进程数目。

4、用 PV 操作实现进程间同步与互斥应注意些什么？

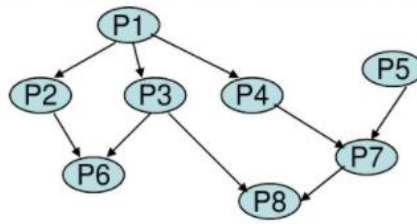
答：(1) 对每一个共享资源（含变量）都要设立信号量，互斥时对一个共享资源设一个信号量，同步时对一个共享资源可能要设多个信号量，视进程间的同步关系而定。(2) 互斥时信号量的初值可大于或等于 1。(3) PV 操作一定要成对调用，互斥时同一进程在临界区前后对同一信号量作 PV 操作，同步时则是不同进程对信号量作 PV 操作，PV 操作的位置一定要正确。(4) 对互斥和同步混合问题，PV 操作可能会嵌套，一般同步的 PV 操作在外，互斥的 PV 操作在内。

三、经典进程通信问题

1、认真看 PPT，掌握 PPT 上介绍过的所有进程同步问题。伪代码填空。

2、用 PV 操作实现任务之间的前驱、后继关系。即，所有前驱任务完成之后，后继任务才能开始执行。

2.1 有 8 个任务，前驱后继关系如下图。P1 任务完成后，可以执行 P2、P3 和 P4 任务；P2 任务和 P3 任务全部执行完毕后，才能够执行 P6 任务。请用 PV 操作控制任务间的前驱后继关系。



```

typedef int Semaphore;
Semaphore s12, s13, s14, s26, s36, s47, s57, s38, s78;
S12= s13=s14=s26=s36=s47=s57=s38=s78=0;

```

<pre> // 进程P1 Void P1() { up(s12); up(s13); up(s14); } </pre>	<pre> // 进程P2 Void P2() { down(s12); up(s26); } </pre>	<pre> // 进程P3 Void P3() { down(s13); up(s36); up(s38); } </pre>	<pre> // 进程P4 Void P4() { down(s14); up(s47); } </pre>
<pre> // 进程P5 Void P5() { up(s57); } </pre>	<pre> // 进程P6 Void P6() { down(s26); down(s36); } </pre>	<pre> // 进程P7 Void P7() { down(s47); down(s57); up(s78); } </pre>	<pre> // 进程P8 Void P8() { down(s38); down(s78); } </pre>

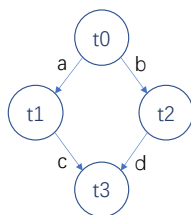
2.2 某分布式程序由 4 个子任务组成，它们分别为：t0、t1、t2 和 t3。任务之间有严格的前驱后继关系：t1 任务在 t0 任务执行完毕后方可执行，t2 任务也是在 t0 任务执行完毕后方可执行，t3 任务在 t1、t2 任务全部执行完毕后方可执行。请（1）参考题目 2.1 中的图，绘图表示任务之间的前驱后继关系（2）设计信号量机制，用 PV 操作控制任务间的前驱后继关系。要求：信号量用得越少越好。并发度越高越好。

4个信号量的解

```

semaphore a, b, c, d;
a.value = b.value = c.value = d.value = 0;

```



```

t0(){
  task t0;
  v(a);
  v(b);
}

t1(){
  p(a);
  task t1;
  v(c);
}

t2(){
  p(b);
  task t2;
  v(d);
}

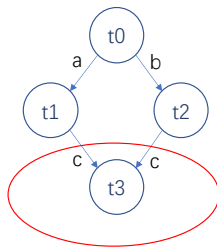
t3(){
  p(c);
  p(d);
  task t3;
}

void main()
{
  parbegin(t0,t1,t2,t3); // 表示启动4个并行/分布式任务
}

```

3个信号量的解

注意，这是错解



```
semaphore a, b, c;
a.value = b.value = 0;
c.value = -1;
```

```
t0(){
    task t0;
    v(a);
    v(b);
}
```

```
t1(){
    p(a);
    task t1;
    v(c);
}
```

```
t2(){
    p(b);
    task t2;
    v(c);
}
```

```
t3(){
    p(c);
    task t3;
}
```

```
void main()
{
    parbegin(t0,t1,t2,t3); // 表示启动4个并行/分布式任务
}
```

如果，t3先运行。
无论c的值是多少，只要小于等于0，一个v操作就会唤醒t3、通过p操作。

3、生产者、消费者问题

3.1 生产者进程互换 P(empty)和 P(mutex) 操作并且消费者进程互换 P(full)和 P(mutex) 操作之后，系统有可能会出现死锁：生产者进程和消费者进程彼此等待，谁也无法前进的僵持情况。试分析系统可能出现死锁的所有情况。

产生死锁的情形 1



```
void producer()
{
    int item;
    while(true)
    {
        item = produceItem();
        p(mutex); // 1 拿到缓存区使用权
        p(empty); // 2 入睡等待空缓存
        buffer[in] = item;
        in=(in+1) mod n;
        v(mutex); // 无法释放缓存使用权
        v(full);
    }
}
```

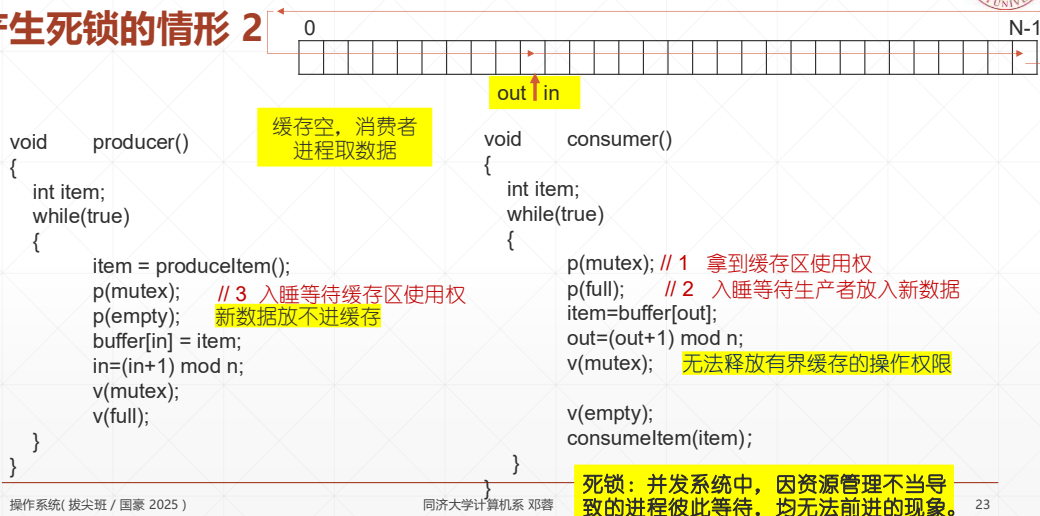
缓存满，生产者进程放新数据

```
void consumer()
{
    int item;
    while(true)
    {
        p(mutex); // 3 入睡等待缓存区使用权
        p(full);
        item=buffer[out];
        out=(out+1) mod n;
        v(mutex);
        v(empty); // 无法释放空缓存
        consumeItem(item);
    }
}
```

死锁：并发系统中，因资源管理不当导致的进程彼此等待，均无法前进的现象。



产生死锁的情形 2



3.2 深刻理解实现。此题有很多变形。

3.3*** 有一个仓库存放两种零件 A 和 B，最大容量各为 m 个。有一车间不断地取 A 和 B 进行装配，每次各取一个。为了避免零件锈蚀，遵循先入库者先出库的原则。有两组供应商分别不断地供应 A 和 B（每次一个），为保证齐套和合理库存，当某种零件的数量比另一种数量少超过 n ($n < m$) 个时，暂停对数量大的零件的进货，集中补充数量少的零件。

解：

```

buffer1, buffer2 : array [0..m-1] of item; //用两个数组模拟 A, B 两个仓库
in1, in2, out1, out2: integer;
in1:=in2:=out1:=out2:=0; //两个仓库的读写指针
var empty1, empty2, full1, full2: semaphore;
empty1:=empty2:=m; //A, B 两个仓库中是否满的同步信号量
full1:=full2:=0; //A, B 两个仓库中是否空的同步信号量

sa, sb: semaphore;
sa:=sb:=n; //两个仓库零件数量差额的同步信号量
//sa.value, 确保 A 零件数量 - B 零件数量 <= n, A 供应商最多还可以往仓库放几个零件, A 的份额
//sb.value, 确保 B 零件数量 - A 零件数量 <= n, B 供应商最多还可以往仓库放几个零件, B 的份额
mutex: semaphore;
mutex:=1; //两个仓库的互斥信号量

cobegin
{
    process producer A // A 供应商不停向 buffer1 中放入 A 零件
    {
        repeat
            P(empty1); // buffer1 是否还有空;
            P(sa); //A 的份额减 1
            P(mutex); //互斥访问临界区
            buffer1[in1] := A 零件;
            in1 := (in1+1) mod m;
    }
}

```

```

        V(mutex);
        V(sb);          //B 的份额加 1
        V(full1);       // buffer1 中多一个 A 零件
    until false;
}

process producer B      //B 供应商不停向 buffer2 中放入 A 零件
{
    repeat
        P(empty2);      // buffer2 是否还有空;
        P(sb);
        P(mutex);       //互斥访问临界区
        Buffer2[in2] := B 零件;
        in2 := (in2+1) mod m;
        V(mutex);
        V(sa);
        V(full2);
    until false;
}

process take            //车间
{
    repeat
        P(full1);
        P(full2);
        P(mutex);       //互斥访问临界区，分别取走两种零件
        Take from buffer1[out1] and buffer2[out2]中的 A、B 零件;
        out1 := (out1+1) mod m;
        out2 := (out2+1) mod m;
        V(mutex);
        V(empty1);
        V(empty2);
        把 A 和 B 装配成产品;
    until false
}
}
coend.

```

4、读者、写者问题

4.1 下面是一个多读者多写者对同一块数据区并发访问的伪代码，阅读代码后回答问题：

```

int readcount = 0;          /* 记录正在读的进程数目*/
semaphore wmutex;
wmutex.value = 1;          /* 对共享对象访问的互斥信号量*/

```

```

void writer()
{
    while(true) {
        p(wmutex);
        WRITEUNIT(); /* 写操作*/
    }
}

```

```

        v(wmutex);
    }
}

void reader()
{
    while(true) {

        readcount++;
        if (readcount == 1)
            p(wmutex);

        READUNIT();

        readcount--;
        if (readcount == 0)
            v(wmutex);

    }
}

```

(1) 请标出上述代码中的临界区。

答：图中高亮部分。

PS: 临界区 critical section。就是好重要的一段代码，不仔细写程序就会出错。所以。。。涉及共享变量并发访问的代码，全是临界区。

黄色高亮一定是临界区。灰色高亮也是的。考研同学，仔细参考考研教程。

(2) 上述代码是否对所有的临界资源都实施了有效的保护，如果没有，请尝试添加必要的信号量，并在代码的适当位置添加对该信号量的 P，V 操作。

答：没有保护好共享变量 readcount。

参考书上，PPT 上的解

(3) 上述代码是否会出现写者饿死的情况，请说明。

答：会。假设当前有进程在读，如果在最后一次读任务完成前，总是有新的读任务到来，那么程序会持续执行读任务，写任务无法执行，最终写者饿死。

(4) 什么是死锁。写出系统产生死锁的 4 个必要条件（必做）。**对照产生死锁的四个必要条件，分析上述写者饿死的情况是系统死锁嘛？

答：背书，参考 PPT。

写者饿死不是死锁。死锁，所有进程阻塞。这里只有写者阻塞。读者正常运行，所以不是死锁。

对照产生死锁的 4 个必要条件。写者只申请 wmutex，没有资源的请求保持；也就不会产生读写双方的循环等待。系统没有死锁。

5、嗜眠理发师

(1) 快餐店有 n 个座位。顾客光临时，若座位全满，转身离开。请问，下面的解正确吗？如果有错，错在哪里。注释：代码中 $seat$ 是空座位的数量。

customer进程

```
semaphore mutex=1;  
int seat=n;
```

```
p(mutex);  
if(seat==0)  
    goto out;  
else  
    seat- -;  
v(mutex);
```

eating;

```
p(mutex);  
seat++;  
v(mutex);
```

```
out:  
exit;
```

操作系统

电信学院计算机系 邓睿

28

不正确。快餐店满的时候，新来的顾客离开前忘记执行 $V(mutex)$ 。
后果是，店里的顾客出不来，外面的顾客进不去。系统死了。

(2) 复印室里有一个操作员为顾客复印资料，有 5 把椅子供顾客休息等待复印。如果没有顾客，则操作员休息。当顾客来到复印室时，如果有空椅子则坐下来，并唤醒复印操作员；如果没有空椅子则必须离开复印室。

(3) 自助复印室有 5 把椅子供顾客休息等待。顾客 FIFS 使用复印机。复印机有人使用，坐在椅子上等待。没有空椅子，转身离开。

